

UT 2 – Ejercicios 16-22 Streams

Ejercicio 16

Vamos a calcular máximo y mínimo de una colección de enteros.

Crea una colección de enteros y, usando `Stream.max(Comparator<? super T> comparator)`, obtén el máximo de la colección, y usando `Stream.min(Comparator<? super T> comparator)`, obtén el mínimo de la colección.

Ten en cuenta:

- Los métodos `min` y `max` necesitan un `Comparator<Integer>` para poder comparar los elementos entre sí y así obtener el máximo y el mínimo.
- Los métodos abstractos de las interfaces funcionales se pueden expresar como una lambda.
- Se puede asignar una lambda a un objeto `Comparator<Integer>` para no tener que repetir la lambda en dos ocasiones.
- Se puede pasar una referencia a método en lugar de crear una lambda.
- Estos métodos devuelven un `Optional<T>`. ¿Qué pasa si la colección está vacía? Puedes usar `Stream.empty`

Prueba distintas formas de realizar esta operación (lambda, referencias a método) y prueba con streams vacíos.

Ejercicio 17

`Stream<T>` es un tipo genérico, y por lo tanto sus métodos `max` y `min` necesitan un comparador. Además, si queremos trabajar con datos numéricos primitivos (`int`, `long`, `double`) usar los wrappers (`Integer`, `Long`, `Double`) provoca cierta sobrecarga.

Hay tres streams especializados: `IntStream`, `LongStream`, `DoubleStream`

Que permiten trabajar con datos primitivos.

Vamos a repetir el ejercicio anterior, pero vamos a:

- Usar el método `ints` de un objeto `Random` para generar streams de números aleatorios. Prueba las diferentes sobrecargas y elige la que consideres más adecuada, y fíjate en qué problemas provocan algunas de ellas.
- Usar los métodos `min()` y `max()` para obtener mínimo y máximo. En este caso, al ser streams especializados, no tenemos que decir a Java como comparar, ya sabe cómo comparar enteros.

Si intentas hacer más de una operación sobre el stream se produce un error. ¿Por qué? Piensa en cómo puedes solucionarlo y cambia el código para que muestre los números generados, luego el mínimo y luego el máximo.

Ejercicio 18

Crea un programa Java y en él:

- Crea una lista aleatoria de 30 nombres.

```
List<String> nombres = List.of(
    "Mario", "Laura", "Alejandro", "Carmen", "Bernardo",
    "Carlos", "Cristina", "David", "Eduardo", "Daniela",
    "Elena", "Oscar", "Enrique", "Fernanda", "Irene",
    "Gabriel", "Beatriz", "Gema", "Hugo", "Julia", "Miguel",
    "Javier", "Ana", "Jorge", "Luis", "Alberto",
    "Guillermo", "Marta", "Nuria", "Francisco");
```

- Usa los métodos `stream()` de la lista y `filter` de `Stream<String>` para crear otra lista que contenga sólo los nombres que comiencen por la letra A. Muestra las dos listas para verificar que no se ha modificado la lista original al crear la segunda.
- Haz pruebas con distintas letras, y prueba también con letras que no devuelvan ningún resultado.
- Vuelve a usar los métodos para obtener una lista con los nombres que sean de una longitud mayor que ocho caracteres. Vuelve a mostrar las dos listas (original y resultado) para comprobar que no se altera la original.

Ejercicio 19

Partiendo de la lista de nombres del ejercicio anterior:

- Usa los métodos `stream()` de la lista y `filter` de `Stream<String>` para crear otra lista que contenga sólo los nombres que contengan cierta letra y que tengan una longitud mayor que cierto valor. Elige la letra y la longitud mínima para que puedan producirse resultados. Tienes que hacerlo de dos formas:
- Primero usando una lambda (como parámetro de `filter`) en la que combinas las dos condiciones (`condicion1 && condición2`)
- Luego usando dos llamadas a métodos `filter` (`filter(lambda1).filter(lambda2)`).
- Muestra las diferentes listas para verificar que la operación es correcta.

Ejercicio 20

Partiendo de la lista de nombres:

- Crea, usando lambdas y un objeto `StringBuilder`, una cadena de caracteres (`String`) que contenga todas las letras iniciales de los nombres de la lista. Muestra la lista y el resultado para verificar que es correcto.

Ejercicio 21

A veces tenemos que convertir un `stream` de un tipo a otro. Por ejemplo, podemos tener un `Stream<Personas>` y convertirlo a un `Stream<Integer>` o a un `IntStream`. Caso de aplicación: queremos obtener todas las edades de un grupo de personas, o queremos la edad máxima de las personas.

Crea un programa que contenga una nueva clase interna llamada “`ResumenPersona`”. Esta clase tendrá:

- Atributos:
 - `nombreApellidos`: `String`
 - `edad`: `int`
- Constructor:

- Un constructor que recibe un objeto de la clase Persona, y que da valor a los atributos de ResumenPersona a partir de los atributos de Persona.
- Métodos:
 - Getters para todos los atributos.

Una vez creada la clase, crea el método principal para que realice las siguientes operaciones:

- Crea una lista de personas.
- Usando el método map, convierte la colección de objetos Persona en un stream de ResumenPersona.
- Muestre los elementos en este stream, uno por línea, utilizando una expresión lambda. Como no hemos creado un toString en ResumenPersona, la expresión lambda será algo más larga que otras que hemos hecho antes.

Ejercicio 22

Crea un programa que:

- Crea una lista aleatoria de personas. Puedes utilizar las del ejercicio anterior.
- Usa los métodos que hemos visto hasta ahora y el método mapToInt, para:
 - Buscar la edad de la persona más mayor de entre los menores de 30 años.
 - Buscar la edad de la persona más joven de entre los mayores de 30 años (incluidos los 30).